

First Individual Assessment



COMS W4156 Advanced Software Engineering (ASE)

First Individual Assessment Fall 2024

Academic Honesty policy: This is an individual assignment. You are not permitted to discuss or work together on the assessment with other students in the class or with any other humans or AI. If you have questions, you may ask the teaching staff, but you may not ask your teammates, other people, search engines, internet forums or AI. Apparent plagiarism, collusion or use of AI (except as explicitly requested in an assessment question) will result in zero credit on the assessment and referral to the relevant disciplinary committee.

Submission policy: Your answers can be as long or as short as you want, but a long meandering answer that incidentally mentions something relevant will receive less credit than a short high-quality answer. Submit your answers all in a single pdf document on courseworks. You can submit as many times as you want until the deadline. Submissions via email, edstem, paper, or any other means besides courseworks will not be graded and will receive zero credit.

Late policy: Submissions posted one day late (which includes everything from 1 minute late to 23 hours 59 minutes late) will receive maximum 75% credit; two days late maximum 50% credit; three days late maximum 25% credit; four days late 0 credit.

Maximum total 100 points. There is no credit for part 0, but 0A is **required** in the sense that we will not grade part 1 (the graded questions) if part 0A is missing or incomplete, and you will receive zero credit overall for the assessment. Part 0B is optional and will not affect your grade.

Ungraded questions: part 0A is required and part 0B is optional.

part 0A (required): Allocate 100 total points among the members of your team, including yourself. Assign some number of points (possibly 0) to each member of your team based on what you believe was that student's relative participation in the team project thus far, and *explain your scores*. Make sure the total is exactly 100. For allocating points, consider the level of effort and contributions for anything relevant to the team project. Note equal participation is not the same thing as equal lines of code produced. Please do not predict the future, consider only the contributions to the team project as of the date of writing. If your team used pair programming with dedicated pairs (not switching off), it is ok to allocate points to pairs rather than individuals. It is very important that you *explain* the scores you give to each student, including yourself: **If you only provide numbers, without meaningful explanations, then your answer will be considered incomplete and you will receive 0 overall on the assessment.**

For example, in a team where all team members participated more or less equally, the scores might be Jisoo 25, Jennie 25, Rosé 25 and Lisa 25, and your explanation would describe who worked on which specific part(s) of the team project effort thus far and what specifically they contributed at a conceptual level (do not tell us about individual functions or lines of code). In a team with a substantial disparity in participation, however, the scores might be Carol 30, Monica 30, Kamala 30 and Nick 10, and your explanation would describe the details of how Carol, Monica and Kamala switched off as needed to more-or-less evenly divide the work, whereas Nick showed up every now and then to give advice and make suggestions but never actually contributed to developing any of the team's deliverables.

part 0B (optional): Please provide feedback on the course so far. Is there anything that you think should have been covered already but wasn't? Is there anything that was covered unnecessarily or should have been put off until later? Is there anything that should have been presented in a different way? Do you think starting with the individual mini-project was helpful, or should we have omitted the mini-project and spread the team project milestones over the entire semester? Is there anything else you would change about the course? Please explain, giving specific examples where warranted. Suggestions for the remainder of this course will be greatly appreciated.

Graded questions: There are three graded questions, parts 1, 2 and 3. Note each question has multiple subparts that should be answered separately.

Part 1 (50 points total):

Imagine the US federal government has imposed a rule that all software accessed over any network in the US must provide a "*backdoor*" that enables the government to monitor all activities of any software clients and/or human users. This rule applies to the service your team is developing for your team project. Monitoring includes the ability to record all transactions between a client and your service including the IP address of the client and the timestamp of the transaction, all requests made by a client to your services, all responses from your service to the client, all data included in requests and responses, and all behaviors performed by your service on behalf of the client.

For the purposes of this assessment, assume your service operates in the US, removing your service from the US market is not feasible, and the clients using your service cannot "opt out" or otherwise evade the rule. [Here !\[\]\(de95854c7ee024cfadc48187bbb781b2_img.jpg\) \(https://techcrunch.com/2024/10/07/the-30-year-old-internet-backdoor-law-that-came-back-to-bite/\)](https://techcrunch.com/2024/10/07/the-30-year-old-internet-backdoor-law-that-came-back-to-bite/) is an example of a similar real-world rule, but you do not need to make your service compliant with any real surveillance or wiretap rule, just the simplified hypothetical rule as described in this assessment.

Think about what new features you would need to add to your service to support the backdoor rule (such as recording IP address and timestamp of every request if you do not already do so) and what existing features you would need to change in some way to support the backdoor rule, while still fulfilling the requirements of this course. These requirements include, but are not limited to, maintaining some relevant data persistently; supporting multiple diverse clients; and providing an API.

Assume your team is required to implement the backdoor for your service and the US government will not modify your codebase or its dependencies in any way. However, the government will review your team's source code and test your team's executable(s) to ensure that your service indeed complies with the rule (even if no real clients ever use your service).

Important note: Do not write or modify any code for this assessment, just think about what coding and testing you would need to do, do not actually do it! Include in your answer a link to your team project github repository but do not change anything in your team's repository in response to this question. Make sure your repo is public or, if private, the instructor and your team's Mentor have collaborator access. Your Mentor will grade the assessments for everyone in your team.

part 1A (15 points): Discuss how your service's *interface* for existing clients would need to change to support the backdoor. Do any new API functions need to be added? Do any existing API functions need to be removed? Does the parameter list of any API function signature need to add parameters, remove parameters or change the type of parameters? Does the set of possible return values (including status or error codes and raised exceptions) for any API function need to change? Or if nothing in your service's interface needs to change to support the backdoor, why not? In other words, how/why are you able to transparently provide the backdoor functionality with no changes to the existing client interface? Please treat the term "API function" as including any kind of entry point to your service, including callbacks and event notifications if your service has any, not just RESTful and RPC function calls.

part 1B (15 points): Discuss how the *implementation* of your service would need to change to support the new backdoor. Make sure you address the implementation changes needed to fulfill any interface changes discussed above as well as any new interfaces you envision for the government to leverage the backdoor. You do not need to provide details of the new government-specific interface but the revised implementation of your service's regular functionality needs to make it easy to develop such an interface.

part 1C (20 points): Describe what specific API, unit and integration *tests* your team would need to add or change/replace to validate whether the backdoor is working as intended without breaking your service's original functionality - which should appear to regular clients to be doing whatever it's supposed to do. Make sure to consider both the interface changes from part 1A and the implementation changes from part 1B.

Part 2 (20 points total):

Both parts of this question concern team development processes and supporting tools. Sadly, something similar really happened many years ago, but the underlying processes and tools may have changed since then.

part 2A (10 points): A student is under the (mistaken!) impression that their grade in a course is based on the number of lines of code they add/change/delete from their team's version control repository. This student copies the source code from a large (irrelevant) project they found online into their team's repository, waits about a week, then deletes that code. A few weeks later, they repeated this behavior

with the code from a different large (also irrelevant) open source project. Near the end of the semester, this student submits "proof" of their team project contributions to their IA using various materials and metrics extracted from their team's repository, where it "looks like" the student indeed edited massive numbers of lines of code on multiple occasions.

Could some member of your team actually do this? State yes, no, or maybe ("it depends"), and explain in detail how your team's processes enable, prevent or mitigate this kind of behavior. If you cannot fathom that anyone on your team would ever do this, imagine instead that a hacker broke into a team member's account. **(Obviously, do not actually try this with your team repository!)**

part 2B (10 points): This student's team became angry with them for messing up their version control repository with the above activities, and there was a heated argument in the team's chat channel. The student then reacted to their teammates' supposed "hostility" by overwriting the team's demo executable with unrelated software, in between when the team practiced their demo late at night, to make sure everything worked, and their actual demo early the next morning for their IA. So the project mysteriously did not work when it came time for the demo, and it took the other team members several hours to figure out what happened and fix it.

Could some member of your team actually do this? State yes, no, or maybe ("it depends"), and explain in detail how your team's processes and tools enable, prevent or mitigate this kind of behavior. If you cannot fathom that anyone on your team would ever do this, imagine instead that a hacker broke into a team member's account. **(Obviously, do not actually try this with your team repository!)**

Part 3 (30 points = 10 points for each of the three API entry points below):

Pick three (3) API entry points from the main branch of your team project where the code implementing that entry point includes conditional branches and/or loops. .

Do not change your codebase to add to or modify your API or its implementation in any way to introduce these properties! If your team project does not already include at least three such API entry points, contact the instructor asap. She will check with your team Mentor and then provide an alternative question if warranted, with extra time to complete.

For each of the three API entry points, describe the valid and invalid input equivalence partitions and how, ideally, you would test each of those partitions including its boundaries, if any. Describe in prose, do not write any new code. If your team already developed and ran any tests that satisfy part or all of these criteria, clarify exactly which tests you are "reusing" from your team's codebase (possibly all or none of them) and which you are inventing "new" for this assessment (possibly all or none of them). Make sure to explain exactly which specific equivalence partitions and/or boundary conditions are targeted by which existing tests already written as code by someone on your team (does not need to be you) and which by your hypothetical new tests.

This is the end of the assessment.

Points 100

Submitting a file upload

File Types pdf

Due	For	Available from	Until
Oct 28, 2024	Everyone	Oct 25, 2024 at 12:01am	Oct 31, 2024 at 11:59pm

+ Create Rubric

🔍 Find Rubric

1A) - 15 pts total

Explain what changes are required to support this backdoor (8 points)

If no changes were made, explain why changes are not required (8 points)

If changes were made:

- -3: Did not describe changes, if needed, to existing APIs
- -2: Did not describe new APIs, if needed
- -3: Did not describe changes to request/response, if needed

Explaining how their code captures the required data points (7 points)

- 1: Did not explain how their code captures client IP currently
- 1: Did not explain how their code captures request timestamp
- 1: Did not explain how their code captures client request
- 1: Did not explain how their code captures client request data
- 1: Did not explain how their code captures server response
- 1: Did not explain how their code captures server response data
- 1: Did not explain how their code captures internal API calls being made by the service for the request

1B) - 15 pts total

- 3: Did not describe the use of a database
- 2: Did not describe the columns, rows
- 3: Did not explain how they plan on logging all actions
- 2: Did not describe the details they need to log
- 3: Did not describe the creation of new APIs
- 2: Did not describe security features

1C) - 20 pts total

Test Coverage and Design (15 points)

Comprehensive description of new or modified API tests to validate the backdoor functionality. (5 points)

- 4: Incomplete or unclear explanation of new/modified API tests if needed.
- 1: Missing discussion on how API tests ensure the backdoor does not interfere with normal client operations.

Detailed explanation of new or modified unit and integration tests needed to ensure the backdoor operates correctly without disrupting existing functionalities. (10 points)

-4: Incomplete or unclear explanation of new/modified unit tests.

-1: Missing discussion on how unit tests ensure the backdoor does not interfere with normal client operations.

-4: Incomplete or unclear explanation of new/modified integration tests.

-1: Missing discussion on how integration tests ensure the backdoor does not interfere with normal client operations.

Consideration of Interface and Implementation Changes (5 points)

-2: Inadequate consideration of interface changes from 1A in described test modifications.

-2: Inadequate consideration of how implementation changes from 1B are validated through testing.

-1: Lack of clarity on how tests verify that implementation changes support both the backdoor and regular service functionality without conflict.

2A) - 10 pts total

-0: Answered why a teammate could or could not do this AND provided a reasonable counter to LoC-grubbing PRs (e.g. code reviews, branch protection rules, etc.).

-5: Answered why a teammate could or could not do this, but did not explain any counters to LoC-grubbing PRs.

-10: No evidence of an appropriate answer.

2B) - 10 pts total

-0: Answered why a teammate could or could not do this AND provided a reasonable counter to malicious PRs (e.g. branch protection rules, code reviews, etc.).

-5: Answered why a teammate could or could not do this, but did not explain any counters to malicious PRs.

-10: No evidence of an appropriate answer

3) 30 pts total, 10 per entry point - individual entry point scored as follows

-0: Correctly described (in enough detail) valid and invalid input equivalence partitions AND correctly described how to test them and their boundaries.

-2: Correctly described (in enough detail) valid and invalid input equivalence partitions AND correctly described how to test them, but did not mention their boundaries (if applicable).

-5: Correctly described (in enough detail) valid and invalid input equivalence partitions but did not adequately describe how to test them and their boundaries.

-10: Did not correctly describe valid and invalid input equivalence partitions and did not correctly describe how to test them and their boundaries.